Mixed models in R: math and syntax for nlme, lme4, and glmmTMB

NRES 746

All three packages maximize a likelihood (or a restricted likelihood) with the random effects integrated out. Fit the same model in two of them and you get the same log-likelihood: the code pairs below agreed on simulated data, except where a fit was singular or a method approximate. Each model is written first in math, then in R, so you can translate in either direction.

Notation

- i indexes observations; j a group (site, stand); k a subgroup nested in j (plot within site); m a second, crossed grouping (observer); t time.
- $j[i]$ is the group that observation i belongs to (the same lookup as `site[i]` in R or Stan).
- βs are fixed effects. Random effects (a_j, b_j, v_{jk}, w_m) are Normal with mean 0 and independent of each other and of the residuals unless stated. $\varepsilon \sim N(0, \sigma_e^2)$ is the residual.
- Every package writes the fixed part the same way (`y ~ x` means $\beta_0 + \beta_1 x$); the random part is where they differ.

In the model	Math	lme4 / glmmTMB term
fixed intercept and slope	$\beta_0 + \beta_1 x_i$	`y ~ x`
random intercept	$+ a_{j[i]},\ \ a_j \sim N(0, \sigma_a^2)$	`(1 | site)`
correlated intercept and slope	$+ a_{j[i]} + b_{j[i]} x_i,\ \ (a_j, b_j) \sim \text{MVN}(\mathbf{0}, G)$	`(x | site)`
uncorrelated intercept and slope	same, with $\text{Cov}(a_j, b_j) = 0$	`(x || site)` (numeric x)
nested groups	$+ a_j + v_{jk}$	`(1 | site/plot)`
crossed groups	$+ a_{j[i]} + w_{m[i]}$	`(1 | site) + (1 | obs)`
AR(1) errors in time	$\eta_t = \phi\, \eta_{t-1} + \varepsilon_t$	`ar1(yearf + 0 | plot)`
log link, Poisson counts	$y_i \sim \text{Pois}(\lambda_i),\ \ \log \lambda_i = \ldots$	`family = poisson`

1. Which package when

- **`glmmTMB`** is the most practical default. One syntax covers Normal, Poisson, binomial, negative binomial, beta, Tweedie and other families; zero inflation; variances that depend on covariates; AR(1) and spatial structures; and crossed random effects. Ecological data often start out looking Poisson and turn out overdispersed or autocorrelated, and here that means changing one argument rather than switching packages. Its defaults are ML fits and Wald tests and intervals; for profile intervals use `confint(fit, method = "profile")` (variance terms come out on the log SD scale, so exponentiate them).
- **`lme4`** (`lmer()`, `glmer()`) is the better choice for inference on Normal models with few groups: `lmer()` fits by REML, `lmerTest` and `pbkrtest` give small-sample (Satterthwaite, Kenward-Roger) tests, and `confint()` profiles the likelihood by default. It's the most widely used and documented, and its sparse-matrix methods handle very large Normal models with crossed effects. No residual correlation or unequal variances.
- **`nlme`** (`lme()`, `gls()`) has the richest menu of residual structures for Normal data (`corCAR1()`, `corSpher()`, `varPower()`, and more), and `gls()` fits them with no random effects at all. Slower, Normal responses only, and crossed random effects are awkward.
- **Bayesian** (`brms`, Stan): priors, small numbers of groups, or models none of these can write.

	nlme	lme4	glmmTMB
Responses	Normal	Normal; glmer: GLMs	many (NB, beta, Tweedie, ...)
Crossed random effects	awkward	yes	yes
Residual correlation	correlation =	no	ar1(), ou(), exp() terms
Unequal variances	weights =	no	dispformula =
Zero inflation	no	no	ziformula =
Default fit	REML	REML (glmer: ML)	ML
Speed on large data	slow	fast	fast
Fixed-effect p-values	t tests	via lmerTest	Wald z

2. Random-effect structures

Write `d` for the data frame; `site`, `plot`, and `obs` (observer) are factors and `x` is numeric.

Random intercept. Observation i in site j:

$$y_{ij} = \beta_0 + \beta_1 x_{ij} + a_j + \varepsilon_{ij}, \qquad a_j \sim N(0, \sigma_a^2)$$

Two observations in the same site share a_j, so their covariance is σ_a^2; observations in different sites are independent. Four parameters: $\beta_0, \beta_1, \sigma_a, \sigma_e$.

```
lme(y ~ x, random = ~ 1 | site, data = d)
lmer(y ~ x + (1 | site), data = d)
glmmTMB(y ~ x + (1 | site), data = d)
```

Correlated random intercept and slope. Each site gets its own line:

$$y_{ij} = (\beta_0 + a_j) + (\beta_1 + b_j)\, x_{ij} + \varepsilon_{ij}, \qquad \begin{pmatrix} a_j \\ b_j \end{pmatrix} \sim \mathrm{MVN}\left(\mathbf{0},\ G = \begin{pmatrix} \sigma_a^2 & \rho_{ab}\sigma_a\sigma_b \\ \rho_{ab}\sigma_a\sigma_b & \sigma_b^2 \end{pmatrix}\right)$$

Six parameters: $\beta_0, \beta_1, \sigma_a, \sigma_b, \rho_{ab}, \sigma_e$. `(x | site)` includes the intercept; `(0 + x | site)` would give a random slope only.

```
lme(y ~ x, random = ~ x | site, data = d)
lmer(y ~ x + (x | site), data = d)
glmmTMB(y ~ x + (x | site), data = d)
```

Uncorrelated random intercept and slope. The same model with $\rho_{ab} = 0$, so G is diagonal (five parameters). In `nlme`, `pdDiag` means "positive-definite, diagonal" (`pdSymm`, the default, is the general form). In `lme4`, `(x || site)` is shorthand for `(1 | site) + (0 + x | site)`.

```
lme(y ~ x, random = list(site = pdDiag(~ x)), data = d)
lmer(y ~ x + (x || site), data = d)
glmmTMB(y ~ x + (x || site), data = d)          # or diag(x | site)
```

If `x` is a factor, `lme4`'s `||` still estimates correlations among the factor's levels. For a diagonal G with a factor, use `diag(trt | site)` in `glmmTMB`, `pdDiag(~ trt)` in `nlme`, or numeric indicator columns in `lme4`: `(1 + trtB + trtC || site)`.

Nested groups. Observation i in plot k within site j:

$$y_{ijk} = \beta_0 + \beta_1 x_{ijk} + a_j + v_{jk} + \varepsilon_{ijk}, \qquad a_j \sim N(0, \sigma_a^2), \quad v_{jk} \sim N(0, \sigma_v^2)$$

The plot effect carries both subscripts: plot 1 in site A is a different plot from plot 1 in site B. `site/plot` expands to `(1 | site) + (1 | site:plot)`, and `site:plot` is the jk subscript. The covariance between two observations is $\sigma_a^2 + \sigma_v^2$ in the same plot, σ_a^2 in the same site but different plots, and 0 in different sites.

```
lme(y ~ x, random = ~ 1 | site/plot, data = d)
lmer(y ~ x + (1 | site/plot), data = d)
glmmTMB(y ~ x + (1 | site/plot), data = d)
```

Crossed groups. Every observer visits several sites:

$$y_i = \beta_0 + \beta_1 x_i + a_{j[i]} + w_{m[i]} + \varepsilon_i, \qquad a_j \sim N(0, \sigma_a^2), \quad w_m \sim N(0, \sigma_w^2)$$

Neither subscript is nested in the other. Two observations covary if they share a site (σ_a^2), an observer (σ_w^2), or both, so no ordering of the data splits V into independent blocks.

```
lmer(y ~ x + (1 | site) + (1 | obs), data = d)
glmmTMB(y ~ x + (1 | site) + (1 | obs), data = d)
d$all <- factor(1)   # nlme: one group holding everything, block-diagonal G (slow for large n)
lme(y ~ x, data = d,
    random = list(all = pdBlocked(list(pdIdent(~ 0 + site), pdIdent(~ 0 + obs)))))
```

3. Residual correlation and unequal variances

Here the error, η, is correlated instead of independent (not available in `lme4`). In `glmmTMB`, correlation enters as a random-effect term with one level per observation; `dispformula = ~0` removes the separate residual variance so the term *is* the residual, matching `nlme`.

AR(1) errors in time. Plot j in year t:

$$y_{jt} = \beta_0 + \beta_1 x_{jt} + \eta_{jt}, \qquad \eta_{jt} = \phi\,\eta_{j,t-1} + \varepsilon_{jt}, \qquad \varepsilon_{jt} \sim N(0, \sigma_e^2)$$

Within a plot, $\mathrm{Cor}(\eta_{js}, \eta_{jt}) = \phi^{|s-t|}$ and $\mathrm{Var}(\eta_{jt}) = \sigma_e^2/(1-\phi^2)$; plots are independent. No random effects, so the model is all residual structure.

```
gls(y ~ x, correlation = corAR1(form = ~ year | plot), data = d)
glmmTMB(y ~ x + ar1(yearf + 0 | plot), dispformula = ~0, data = d)     # yearf <- factor(year)
```

Random site intercept plus AR(1) within each plot. Plot k in site j, year t:

$$y_{jkt} = \beta_0 + \beta_1 x_{jkt} + a_j + \eta_{jkt}, \qquad a_j \sim N(0, \sigma_a^2), \qquad \eta_{jkt} \text{ AR(1) within plot}$$

$V = \sigma_a^2 ZZ^T + \sigma_e^2 R$: the random intercept adds σ_a^2 wherever two values share a site, and R holds the AR(1) correlations within each plot.

```
lme(y ~ x, random = ~ 1 | site, correlation = corAR1(form = ~ year | site/plot), data = d)
glmmTMB(y ~ x + (1 | site) + ar1(yearf + 0 | plot), dispformula = ~0, data = d)  # unique plot labels
```

Gaps in time: the Ornstein-Uhlenbeck process. The continuous-time version of AR(1) uses the actual times:

$$\mathrm{Cor}(\eta_{js}, \eta_{jt}) = e^{-\theta|s-t|} = \phi^{|s-t|}, \qquad \phi = e^{-\theta}$$

so a two-year gap gets ϕ^2. `glmmTMB`'s `ar1()` treats the *observed* times as consecutive (a missing year counts as a lag of 1); `ou()` with `numFactor()` times handles gaps. `nlme`'s `corAR1()` handles gaps with integer times; `corCAR1()` takes non-integer times.

```
gls(y ~ x, correlation = corAR1(form = ~ year | plot), data = d)      # handles gaps
glmmTMB(y ~ x + ou(tt + 0 | plot), dispformula = ~0, data = d)       # tt <- numFactor(year)
```

Exponential spatial correlation. Locations i and k in the same site, a distance d_{ik} apart:

$$y_i = \beta_0 + \beta_1 x_i + \eta_i, \qquad \mathrm{Cor}(\eta_i, \eta_k) = e^{-d_{ik}/\ell}$$

where ℓ is the range: correlation falls to $e^{-1} \approx 0.37$ at distance ℓ.

```
gls(y ~ x, correlation = corExp(form = ~ east + north | site), data = d)
glmmTMB(y ~ x + exp(pos + 0 | site), dispformula = ~0, data = d)     # pos <- numFactor(east, north)
```

A separate residual variance for each treatment.

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \qquad \varepsilon_i \sim N\left(0, \sigma^2_{\mathrm{trt}[i]}\right)$$

```
gls(y ~ x, weights = varIdent(form = ~ 1 | trt), data = d)
glmmTMB(y ~ x, dispformula = ~ trt, data = d)
```

Other `nlme` options: `corCAR1()` (continuous time), `corCompSymm()`, `corGaus()`, `corSpher()`; `varPower(form = ~ fitted(.))`, `varExp(form = ~ x)`. Other `glmmTMB` options: `cs()` (compound symmetry), `gau()`, `mat()` (Matern), `us()` and `diag()` for random-effect covariances. See `?corClasses`, `?varClasses`, and `vignette("covstruct", package = "glmmTMB")`.

4. Non-Normal responses (GLMMs)

A GLMM puts the random effects on the link scale, where they are Normal, and uses a non-Normal distribution for each observation given those effects. `nlme` fits none of these.

Poisson counts.

$$y_{ij} \sim \mathrm{Pois}(\lambda_{ij}), \qquad \log \lambda_{ij} = \beta_0 + \beta_1 x_{ij} + a_j, \qquad a_j \sim N(0, \sigma_a^2)$$

```
glmer(count ~ x + (1 | site), family = poisson, data = d)
glmmTMB(count ~ x + (1 | site), family = poisson, data = d)
```

Successes s out of n trials.

$$s_{ij} \sim \mathrm{Binom}(n_{ij}, p_{ij}), \qquad \mathrm{logit}(p_{ij}) = \beta_0 + \beta_1 x_{ij} + a_j$$

```r
glmer(cbind(s, n - s) ~ x + (1 | site), family = binomial, data = d)
glmmTMB(cbind(s, n - s) ~ x + (1 | site), family = binomial, data = d)
```

Overdispersed counts. Three common choices:

- Negative binomial (`nbinom2`): $\mathrm{Var}(y_{ij}) = \mu_{ij} + \mu_{ij}^2/\theta$ with $\log \mu_{ij} = \beta_0 + \beta_1 x_{ij} + a_j$. (`nbinom1`: $\mathrm{Var} = \mu(1+\phi)$.)
- Observation-level random effect: add $o_i \sim N(0, \sigma_o^2)$ to the Poisson's $\log \lambda$, one per observation (`obs_id` has a unique level for each row).
- Zero-inflated negative binomial: $y_{ij} = 0$ with probability π; otherwise y_{ij} is negative binomial as above. `ziformula = ~ 1` means a constant π (on the logit scale); `~ x` would let it vary.

```r
glmer.nb(count ~ x + (1 | site), data = d)                    # slower, approximate
glmmTMB(count ~ x + (1 | site), family = nbinom2, data = d)
glmer(count ~ x + (1 | site) + (1 | obs_id), family = poisson, data = d)
glmmTMB(count ~ x + (1 | site) + (1 | obs_id), family = poisson, data = d)
glmmTMB(count ~ x + (1 | site), family = nbinom2, ziformula = ~ 1, data = d)
# others: beta_family() for proportions in (0, 1), tweedie(), Gamma(link = "log"), ...
```

Offsets for effort or area. A count from a plot of area A_i: $\log \lambda_i = \log A_i + \beta_0 + \beta_1 x_i + a_{j[i]}$, so λ_i / A_i is the density. Add `+ offset(log(area))` to the formula in any package.

5. Fitting options

	nlme	lme4	glmmTMB
REML/ML (default 1st)	`method = "REML" / "ML"`	`REML = TRUE / FALSE (lmer)`	`REML = FALSE / TRUE`
GLMM integration	--	`Laplace; nAGQ = (one effect)`	`Laplace`
Spot a bad fit	--	`isSingular(fit)`	`diagnose(fit)`

Convergence warnings? Try another optimizer:

```r
lme(..., control = lmeControl(opt = "optim"))
lmer(..., control = lmerControl(optimizer = "bobyqa"))
allFit(fit)                                  # refit an lme4 model with every optimizer and compare
glmmTMB(..., control = glmmTMBControl(optimizer = optim, optArgs = list(method = "BFGS")))
```

- REML gives less biased variance estimates. Use ML when comparing models with different fixed effects (by AIC or a likelihood ratio test). `lmer()` and `glmmTMB()` have different defaults, so their default fits of the same Normal model differ slightly.
- `anova(fit1, fit2)` gives a likelihood ratio test in all three. Testing a variance at zero puts it on a boundary, so that p-value is conservative (too large).
- A **singular fit** (a variance estimated at zero, or a correlation at ± 1) means the data can't support that random-effect structure. Simplify it (drop the correlation, or the random slope) rather than trusting the estimates. Different packages may stop at slightly different points on such a flat surface.
- Center and rescale predictors before blaming the optimizer.

6. Getting results out

	nlme	lme4	glmmTMB
Fixed effects	`fixef(fit)`	`fixef(fit)`	`fixef(fit)$cond`
Group deviations	`ranef(fit)`	`ranef(fit)`	`ranef(fit)`
Group coefficients	`coef(fit)`	`coef(fit)`	`coef(fit)`
Variance components	`VarCorr(fit)`	`VarCorr(fit)`	`VarCorr(fit)`
Confidence intervals	`intervals(fit)`	`confint(fit) (profile)`	`confint(fit) (Wald)`
Mean predictions	`predict(fit, level = 0)`	`predict(fit, re.form = NA)`	`predict(fit, re.form = NA)`
GLMM response scale	--	`type = "response"`	`type = "response"`
Simulate new data	--	`simulate(fit)`	`simulate(fit)`
Residual checks	`ACF(fit, resType = "n")`	DHARMa (below)	DHARMa (below)

In the math: `fixef()` returns the βs; `ranef()` the predicted a_j, b_j (shrunk toward 0); `coef()` each group's $\beta_0 + a_j$ and $\beta_1 + b_j$; `VarCorr()` the σs and ρ_{ab}.

- `summary()`, `logLik()`, `AIC()`, and `nobs()` work everywhere. Log-likelihoods are comparable across packages for the same data and the same estimation method (ML vs. REML).
- `glmmTMB` keeps separate parts for the mean (`$cond`), zero inflation (`$zi`), and dispersion (`$disp`).
- For GLMMs, ordinary residual plots are hard to read. The `DHARMa` package simulates from the fitted model and gives residuals that should be uniform if the model is right.
- `broom.mixed::tidy(fit)` returns a data frame of estimates from any of the three.

7. Things that trip people up

- **Names for the same thing.** The AR(1) coefficient ϕ is `Phi` in `nlme` and appears as a correlation labeled `(ar1)` in `glmmTMB`'s `VarCorr()`. With AR(1) errors, both report the SD of the total error, $\sigma_e/\sqrt{1-\phi^2}$, not the innovation SD σ_e.
- **Time and space in `glmmTMB`.** `ar1()` needs a factor of equally spaced times and treats the observed times as consecutive. Use `ou()` with `numFactor()` times when there are gaps.
- **Unique times.** Each `time | group` combination must appear once. Two observations in the same plot and year need an extra grouping level.
- **Factors.** Make grouping variables factors. `glmmTMB` time terms need a factor (`ar1()`) or `numFactor()` (`ou()`, `exp()`). A factor inside `||` doesn't give uncorrelated effects (section 2).
- **Nested labels.** If plots are numbered 1, 2, 3 within every site, `(1 | plot)` wrongly treats plot 1 at every site as one plot. Use `(1 | site/plot)` or give plots unique labels.
- **Missing values** stop `nlme` fits; add `na.action = na.omit`.
- **p-values.** `lmer()` reports none by design (the degrees of freedom are unclear). `lmerTest` adds approximate ones; with few groups, treat any of them as rough.
- **Few groups.** With fewer than about five levels, a random-effect variance is poorly estimated in every package. Consider fixed effects or a Bayesian fit with an informative prior.

Learn more: Bolker's GLMM FAQ, https://bbolker.github.io/mixedmodels-misc/glmmFAQ.html; Bates et al. (2015, *J. Stat. Software* 67(1)) for `lme4`; Brooks et al. (2017, *R Journal* 9: 378-400) and the `glmmTMB` vignettes; Pinheiro and Bates (2000), *Mixed-Effects Models in S and S-PLUS*; Zuur et al. (2009), *Mixed Effects Models and Extensions in Ecology with R.*